Gramática del Lenguaje EsJS

Esteban Barrera Sanabria

---

0. Símbolos del Usuario

Los siguientes símbolos son **terminales** proporcionados por el **lexer** (usuario no escribe reglas para ellos):

- `<id>`: Identificador (variable, función, etc). Ejemplo: `x`, `contador`, `_privado`, `$valor`
- `<cadena_de_caracteres>`: Literal de cadena entre comillas. Ejemplo: `"hola"`, `'mundo'`
- `<valor_numerico>`: Literal numérico. Ejemplo: `42`, `3.14`, `.5`
- `<final de archivo>`: Indica el final del codigo fuente.

---

1. Programa y Sentencias

Un programa es una o más sentencias seguidas de EOF.

```
Programa → Sentencias EOF
```

```
Sentencias → Sentencia OtrasSentencias
```

```
EOF → <final de archivo>
```

```
OtrasSentencias → PyC Sentencia OtrasSentencias | ε
```

```
Sentencia → SentenciaDeclaracionVariable | SentenciaExpresion |
SentenciaFuncion | Bloque
| SentenciaControlFlujo | SentenciaClase | SentenciaIntento |
SentenciaRetorno | SentenciaRota
| SentenciaSiSinElse
```

```
PyC → ";" | ε
```

---

2. Sentencia de Declaración de Variables

```
SentenciaDeclaracionVariable → TipoVariable <id> DeclaracionResto
```

```
TipoVariable → "var" | "const" | "mut"
```

```
DeclaracionResto → "=" Expresion OtrasDeclaracionesSeguidas | ε
```

```
OtrasDeclaracionesSeguidas → "," <id> "=" Expresion
OtrasDeclaracionesSeguidas | ε
```

---

3. Sentencia de Expresión

```
SentenciaExpresion → ParamFuncionFlecha | ExpresionRegular
```

```
ParamFuncionFlecha → <id> "=>" FormaFuncion | "(" ParametrosFuncionFlecha
")" "=>" FormaFuncion
```

```
ParametrosFuncionFlecha → Parametros | ε
```

```
ExpresionRegular → Expresion (OperadorAsignacion Expresion)?
```

```
OperadorAsignacion → "=" | "+=" | "-=" | "*=" | "/=" | "%=" | "**="
```

---

4. Funciones

```
SentenciaFuncion → "funcion" <id> "(" ParametrosOpcionales ")" Bloque
```

```
ParametrosOpcionales → Parametros | ε
```

```
FuncionAnonima → "funcion" "(" ParametrosOpcionales ")" Bloque
```

```
FormaFuncion → Bloque | Expresion
```

```
Parametros → <id> OtrosParametros
```

```
OtrosParametros → "," <id> OtrosParametros | ε
```

```
Argumentos → ListaExpresiones | ε
```

---

5. Control de Flujo y Bloques

```
SentenciaControlFlujo → SentenciaSi | SentenciaMientras | SentenciaHacer
  | SentenciaPara
```

```
Bloque → "{" Sentencias "}"
```

```
SentenciaSi → SentenciaSiSinElse | SentenciaSiConElse
```

```
SentenciaSiSinElse → "si" "(" Expresion ")" BloqueOSentenciaSinElse
```

```
SentenciaSiConElse → "si" "(" Expresion ")" BloqueOSentenciaConElse "sino"
(SentenciaSiConElse | Bloque)
```

```
BloqueOSentenciaSinElse → Bloque | SentenciaNoSi
```

```
BloqueOSentenciaConElse → Bloque | Sentencia
```

```
SentenciaNoSi → SentenciaDeclaracionVariable | SentenciaExpresion |
SentenciaFuncion
| SentenciaControlFlujoNoSi | SentenciaClase | SentenciaIntento |
SentenciaRetorno | SentenciaRota
```

```
SentenciaControlFlujoNoSi → SentenciaMientras | SentenciaHacer |
SentenciaPara
```

```
SentenciaMientras → "mientras" "(" Expresion ")" BloqueSentencia
```

```
SentenciaHacer → "hacer" BloqueSentencia "mientras" "(" Expresion ")"
```

```
SentenciaPara → ParaClasico | ParaDeEn
```

```
ParaClasico → "para" "(" Inicializador ";" Condicion ";" Actualizacion ")"
BloqueSentencia
```

```
Inicializador → SentenciaDeclaracionVariable | Expresion | ε
```

```
Actualizacion → Expresion | ε
```

```
ParaDeEn → "para" "(" SentenciaDeclaracionVariable FormaParaDeEn Expresion
")" BloqueSentencia
```

```
FormaParaDeEn → "de" | "en"
```

---

6. Otras Sentencias

```
SentenciaElegir → "elegir" "(" Expresion ")" "{" Casos "}"
```

```
Casos → ListaDeCasos PorDefecto
```

```
ListaDeCasos → Caso ListaCasos | ε
```

```
Caso → "caso" Expresion ":" Sentencias
```

```
PorDefecto → "porDefecto" ":" Sentencias | ε
```

```
SentenciaIntento → "intentar" Bloque "capturar" CapturaClausula
 Bloque FinalmenteOpcional
```

```
FinalmenteOpcional → "finalmente" Bloque | ε
```

```
CapturaClausula → ("("<id>")") | ε
```

```
SentenciaRetorno → TipoRetorno RetornoOpcional
```

```
TipoRetorno → "retornar" | "lanzar"
```

```
RetornoOpcional → Expresion | ε
```

```
SentenciaRota → "romper" | "continuar"
```

```
SentenciaClase → "clase" <id> HerenciaOpcional "{" Miembros "}"
```

```
HerenciaOpcional → "extiende" <id> | ε
```

```
Miembros → MiembroClase Miembros | ε
```

```
MiembroClase → ("constructor" | NombrePropiedad) "(" ParametrosOpcionales
")"
  Bloque | NombrePropiedad "=" Expresion
```

---

7. Expresiones

7.1. Jerarquía de Precedencia

Las expresiones se evalúan por niveles de precedencia (de menor a mayor):

```
Expresion → ExprTernaria
```

```
ExprTernaria → ExprLogica | ExprLogica "?" Expresion ":" Expresion
```

```
ExprLogica → ExprComparacion OtrasExprLogica
```

```
OtrasExprLogica → OperadorLogica ExprComparacion OtrasExprLogica | ε
```

```
OperadorLogica → "&&" | "||"
```

```
ExprComparacion → ExprAritmetica OtrasExprComparacion
```

```
OtrasExprComparacion → OperadorComparacion ExprAritmetica
OtrasExprComparacion | ε
```

```
OperadorComparacion → "==" | "!=" | "===" | "!==" | "<" | ">" | "<=" | ">="
| "instanciaDe"
```

```
ExprAritmetica → ExprMultiplicativa OtrasExprAritmeticas
```

```
OtrasExprAritmeticas → OperadorAritmetico ExprMultiplicativa
OtrasExprAritmeticas | ε
```

```
OperadorAritmetico → "+" | "-"
```

```
ExprMultiplicativa → ExprUnaria OtrasExprMultiplicativas
```

```
OtrasExprMultiplicativas → OperadorMultiplicativo ExprMultiplicativa
OtrasExprMultiplicativas | ε
```

```
OperadorMultiplicativo → "*" | "/" | "%" | "**"
```

```
ExprUnaria → OperadorUnarioOpcional ExprPostfija
```

```
OperadorUnarioOpcional → OperadorUnario | ε
```

```
OperadorUnario → "+" | "-" | "!" | "++" | "--" | "tipoDe" | "vacio"
   | "eliminar" | "esperar"
```

```
ExprPostfija → Primaria OtrasPostfijo
```

```
OtrasPostfijo → Postfijo OtrasPostfijo | ε
```

```
Postfijo → "." Propiedad | "[" Expresion "]" | "(" Argumentos ")" | "++" |
"--"
```

7.2. Expresión Primaria

```
Primaria → <id> | <valor_numerico> | <cadena_de_caracteres> | "verdadero" |
"falso" | "nulo"
  | "indefinido" | "Infinito" | "NuN" | Builtin | Crear | "(" Expresion ")"
  | FuncionAnonima | ArregloLiteral | ObjetoLiteral
```

```
Builtin → "Arreglo" | "Booleano" | "Cadena" | "Fecha" | "Funcion" |
"Matriz"
| "Numero" | "consola" | "Mate" | "NuN" | "Infinito"
```

```
Crear → "crear" Constructor ("(" Argumentos ")")?
```

```
Constructor → <id> | "Arreglo" | "Booleano" | "Cadena" | "Fecha" |
"Funcion" | "Matriz"
  | "Numero"
```

7.3. Arreglos y Objetos

```
ArregloLiteral → "[" Argumentos "]"
```

```
ListaExpresiones → Expresion ListaExpresionesResto ComaOpcional
```

```
ListaExpresionesResto → "," Expresion ListaExpresionesResto | ε
```

```
ComaOpcional → "," | ε
```

```
ObjetoLiteral → "{" PropiedadObjeto ListaPropiedades ComaOpcional "}"
```

```
ListaPropiedades → "," PropiedadObjeto ListaPropiedades | ε
```

```
PropiedadObjeto → NombrePropiedad | NombrePropiedad ":" Expresion
  | NombrePropiedad "(" ParametrosOpcionales ")" Bloque
```

```
NombrePropiedad → <id> | <cadena_de_caracteres> | <valor_numerico>
```

Declaración de Variable Simple

Código:

```
const x = 10
```

Árbol de derivación:

```
Programa
  ├─ Sentencias
  │   └─ Sentencia OtrasSentencias
  │       ├─ SentenciaDeclaracionVariable
  │       │   ├─ TipoVariable → "const"
  │       │   ├─ <id> → x
```

```
    │       │   └─ DeclaracionResto
    │       │       ├─ "="
    │       │       ├─ Expresion
    │       │       │   └─ ExprTernaria
    │       │       │       └─ ExprLogica
    │       │       │           └─ ExprComparacion
    │       │       │               └─ ExprAritmetica
    │       │       │                   └─ ExprMultiplicativa
    │       │       │                       └─ ExprUnaria
    │       │       │                           └─ ExprPostfija
    │       │       │                               └─ Primaria →
<valor_numerico> (10)
    │       │       └─ OtrasDeclaracionesSeguidas → ε
    │       └─ OtrasSentencias → ε
    └─ EOF
```

Sentencia Si con Else

Código:

```
si (x > 0) { consola.escribir("positivo") } sino { consola.escribir("otro")
}
```

Árbol de derivación (simplificado):

```
Programa
  ├─ Sentencias
  │   └─ Sentencia
  │       └─ SentenciaControlFlujo
  │           └─ SentenciaSi
  │               ├─ "si"
  │               ├─ "(" Expresion (x > 0) ")"
  │               ├─ BloqueSentencia
  │               │   └─ Bloque
  │               │       └─ Sentencias
  │               │           └─ Sentencia
  │               │               └─ SentenciaExpresion
(consola.escribir...)
  │               └─ Sino
  │                   ├─ "sino"
  │                   └─ SinoResto
  │                       └─ Bloque
  │                           └─ Sentencias
  │                               └─ Sentencia
  │                                   └─ SentenciaExpresion
(consola.escribir...)
  └─ EOF
```

Función Declarada

Código:

```
funcion cuadrado(x) { retornar x * x }
```

Árbol de derivación (simplificado):

```
Programa
  ├─ Sentencias
  │   └─ Sentencia
  │       └─ SentenciaFuncion
  │           ├─ "funcion"
  │           ├─ <id> → cuadrado
  │           ├─ "("
  │           ├─ ParametrosOpcionales
  │           │   └─ Parametros
  │           │       └─ <id> → x
  │           │           └─ OtrosParametros → ε
  │           ├─ ")"
  │           └─ Bloque
  │               └─ "{"
  │               ├─ Sentencias
  │               │   └─ Sentencia
  │               │       └─ SentenciaRetorno
  │               │           ├─ TipoRetorno → "retornar"
  │               │           └─ RetornoOpcional
  │               │               └─ Expresion (x * x)
  │               └─ "}"
  └─ EOF
```

Gramática Completa EBNF

```
Programa → Sentencias EOF

Sentencias → Sentencia OtrasSentencias

EOF → <final de archivo>

OtrasSentencias → PyC Sentencia OtrasSentencias | ε

PyC → ";" | ε

Sentencia →
  SentenciaDeclaracionVariable
  | SentenciaExpresion
  | SentenciaFuncion
```

```
    | Bloque
    | SentenciaControlFlujo
    | SentenciaClase
    | SentenciaIntento
    | SentenciaRetorno
    | SentenciaRota
    | SentenciaSiSinElse

SentenciaDeclaracionVariable → TipoVariable <id> DeclaracionResto

TipoVariable → "var" | "const" | "mut"

DeclaracionResto → "=" Expresion OtrasDeclaracionesSeguidas | ε

OtrasDeclaracionesSeguidas → "," <id> "=" Expresion
OtrasDeclaracionesSeguidas | ε

SentenciaExpresion → ParamFuncionFlecha | ExpresionRegular

ParamFuncionFlecha → <id> "=>" FormaFuncion | "(" ParametrosFuncionFlecha
")" "=>" FormaFuncion

ParametrosFuncionFlecha → Parametros | ε

ExpresionRegular → Expresion (OperadorAsignacion Expresion)?

OperadorAsignacion → "=" | "+=" | "-=" | "*=" | "/=" | "%=" | "**="

OtraPostfijo → Postfijo OtraPostfijo | ε

AsignacionOpcional → OperadorAsignacion Expresion | ε

OperadorAsignacion → "=" | "+=" | "-=" | "*=" | "/=" | "%=" | "**="

SentenciaFuncion → "funcion" <id> "(" ParametrosOpcionales ")" Bloque

ParametrosOpcionales → Parametros | ε

FuncionAnonima → "funcion" "(" ParametrosOpcionales ")" Bloque

FormaFuncion → Bloque | Expresion

Parametros → <id> OtrosParametros

OtrosParametros → "," <id> OtrosParametros | ε

Argumentos → ListaExpresiones | ε

SentenciaControlFlujo → SentenciaSi | SentenciaMientras | SentenciaHacer |
SentenciaPara

Bloque → "{" Sentencias "}"

SentenciaSi → SentenciaSiSinElse | SentenciaSiConElse
```

```
SentenciaSiSinElse → "si" "(" Expresion ")" BloqueOSentenciaSinElse

SentenciaSiConElse → "si" "(" Expresion ")" BloqueOSentenciaConElse "sino"
(SentenciaSiConElse | Bloque)

BloqueOSentenciaSinElse → Bloque | SentenciaNoSi

BloqueOSentenciaConElse → Bloque | Sentencia

SentenciaNoSi →
  SentenciaDeclaracionVariable
  | SentenciaExpresion
  | SentenciaFuncion
  | SentenciaControlFlujoNoSi
  | SentenciaClase
  | SentenciaIntento
  | SentenciaRetorno
  | SentenciaRota

SentenciaControlFlujoNoSi → SentenciaMientras | SentenciaHacer |
SentenciaPara

SentenciaMientras → "mientras" "(" Expresion ")" BloqueSentencia

BloqueSentencia → Bloque | Sentencia

SentenciaHacer → "hacer" BloqueSentencia "mientras" "(" Expresion ")"

SentenciaPara → ParaClasico | ParaDeEn

ParaClasico → "para" "(" Inicializador ";" Condicion ";" Actualizacion ")"
BloqueSentencia

Inicializador → SentenciaDeclaracionVariable | Expresion | ε

Condicion → Expresion | ε

Actualizacion → Expresion | ε

ParaDeEn → "para" "(" SentenciaDeclaracionVariable FormaParaDeEn Expresion
")" BloqueSentencia

FormaParaDeEn → "de" | "en"

SentenciaElegir → "elegir" "(" Expresion ")" "{" Casos "}"

Casos → ListaDeCasos PorDefecto

ListaDeCasos → Caso ListaDeCasos | ε

Caso → "caso" Expresion ":" Sentencias

PorDefecto → "porDefecto" ":" Sentencias | ε
```

```
SentenciaIntento → "intentar" Bloque "capturar" CapturaClausula Bloque
FinalmenteOpcional

CapturaClausula → ("(" <id> ")")?

FinalmenteOpcional → "finalmente" Bloque | ε

SentenciaRetorno → TipoRetorno RetornoOpcional

TipoRetorno → "retornar" | "lanzar"

RetornoOpcional → Expresion | ε

SentenciaRota → "romper" | "continuar"

SentenciaClase → "clase" <id> HerenciaOpcional "{" Miembros "}"

HerenciaOpcional → "extiende" <id> | ε

Miembros → MiembroClase Miembros | ε

MiembroClase → ("constructor" | NombrePropiedad) "(" ParametrosOpcionales
")" Bloque
  | NombrePropiedad "=" Expresion

Expresion → ExprTernaria

ExprTernaria → ExprLogica | ExprLogica "?" Expresion ":" Expresion

ExprLogica → ExprComparacion OtrasExprLogica

OtrasExprLogica → OperadorLogica ExprComparacion OtrasExprLogica | ε

OperadorLogica → "&&" | "||"

ExprComparacion → ExprAritmetica OtrasExprComparacion

OtrasExprComparacion → OperadorComparacion ExprAritmetica
OtrasExprComparacion | ε

OperadorComparacion → "==" | "!=" | "===" | "!==" | "<" | ">" | "<=" | ">="
| "instanciaDe"

ExprAritmetica → ExprMultiplicativa OtrasExprAritmeticas

OtrasExprAritmeticas → OperadorAritmetico ExprMultiplicativa
OtrasExprAritmeticas | ε

OperadorAritmetico → "+" | "-"

ExprMultiplicativa → ExprUnaria OtrasExprMultiplicativas

OtrasExprMultiplicativas → OperadorMultiplicativo ExprMultiplicativa
```

```
OtrasExprMultiplicativas | ε

OperadorMultiplicativo → "*" | "/" | "%" | "**"

ExprUnaria → OperadorUnarioOpcional ExprPostfija

OperadorUnarioOpcional → OperadorUnario | ε

OperadorUnario → "+" | "-" | "!" | "++" | "--" | "tipoDe" | "vacio" |
"eliminar" | "esperar"

ExprPostfija → Primaria OtrasPostfijo

OtrasPostfijo → Postfijo OtrasPostfijo | ε

Postfijo → "." Propiedad
  | "[" Expresion "]"
  | "(" Argumentos ")"
  | "++"
  | "--"

Primaria →
  <id>
  | <valor_numerico>
  | <cadena_de_caracteres>
  | "verdadero"
  | "falso"
  | "nulo"
  | "indefinido"
  | "Infinito"
  | "NuN"
  | Builtin
  | Crear
  | "(" Expresion ")"
  | FuncionAnonima
  | ArregloLiteral
  | ObjetoLiteral

Builtin →
  "Arreglo" | "Booleano" | "Cadena" | "Fecha" | "Funcion" | "Matriz"
  | "Numero" | "consola" | "Mate" | "NuN" | "Infinito"

Crear → "crear" Constructor ("(" Argumentos ")")?

Constructor →
  <id>
  | "Arreglo"
  | "Booleano"
  | "Cadena"
  | "Fecha"
  | "Funcion"
  | "Matriz"
  | "Numero"
```

```
ArregloLiteral → "[" ListaExpresiones "]"

ListaExpresiones → Expresion ListaExpresionesResto ComaOpcional

ListaExpresionesResto → "," Expresion ListaExpresionesResto | ε

ComaOpcional → "," | ε

ObjetoLiteral → "{" PropiedadObjeto ListaPropiedades ComaOpcional "}"

ListaPropiedades → "," PropiedadObjeto ListaPropiedades | ε

PropiedadObjeto →
  NombrePropiedad
  | NombrePropiedad ":" Expresion
  | NombrePropiedad "(" ParametrosOpcionales ")" Bloque

NombrePropiedad → <id> | <cadena_de_caracteres> | <valor_numerico>
```